CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 2

CO2: Analyze software requirements to design scalable and efficient system architectures using collaborative tools like Jira and Confluence, along with structured modeling techniques such as UML diagrams and architectural design patterns.

Assignment Title: Design and Validate Requirements and Architecture for the Omnichannel Order Fulfilment Platform

Nature of Assignment: Individual

Name: Dharshavardhan.SB

Reg: 192420039

Duration: One week

Total Marks: 50

Submission Date: _____

Scenario — Order Fulfilment Requirements and Scalable Architecture

A retailer needs an order fulfilment architecture that connects checkout, stock reservation, payment callbacks, warehouse picking, courier tracking and refunds without losing consistency or exposing restricted customer information. Repeated payment callbacks, two buyers competing for the last unit and failed courier synchronization create conflicting requirements for speed, availability, accountability and confidentiality. Analyze stakeholder needs and create a testable requirements specification with traceable UML models and justified architectural alternatives. Validate order and refund invariants, data ownership and failure handling. Redesign the selected architecture for additional regions, increased demand and disconnected warehouse scanners while explaining consistency, availability and operating-cost trade-offs. The retailer proposes an omnichannel order fulfilment platform. Its intended modules are a storefront catalogue, cart and checkout, payment integration, inventory synchronization, order tracking, a warehouse picking dashboard, returns and refunds, a support console and sales analytics. The system should connect a customer order to stock reservation, payment confirmation, picking, dispatch, tracking, delivery and, where needed, an authorized return and refund.

Architecture conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Checkout, stock reservation, payment confirmation, picking, dispatch, tracking, returns and refunds
Users	Administrator, picker, support agent, finance reviewer, store manager and customer

Observation	Scenario planning condition
Reference data	Synthetic products, orders and stock movements; no real customer or card data
Integration constraint	Payment gateway or courier API may be unavailable or may send duplicate callbacks; preserve local order records and retry safely
Validation need	Verify stock, order status, price, authorization, persistence and refund audit behaviour

Principal users are enterprise administrators, warehouse pickers, support agents, finance reviewers, store managers and customers. They need different permissions. Pickers require accurate pick lists; support agents require readable order and return status; finance reviewers require auditable refund approvals; engineers require traceable links between orders, payments and courier shipments. Automation must preserve human responsibility for refund approval and stock corrections.

Customers want fast checkout and clear tracking; pickers want reliable stock reservation; support agents want order visibility without seeing payment data; finance reviewers require approval before refunds above a limit. Management wants consolidated sales, stock and refund reporting. These interests may conflict when speed, privacy and refund controls compete.

Use a manageable representative release and synthetic orders and stock events. Jira, Confluence, GitHub, Jenkins and Docker are suitable reference tools; select and justify alternatives where appropriate. A React interface, FastAPI service and PostgreSQL database are one feasible reference architecture, not compulsory choices. External integrations may be demonstrated through controlled test interfaces when live access is unavailable, with their status disclosed.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, stock counts and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live inventory control or secure payment handling. Verify claims through repeatable checks. Distinguish deterministic rules, such as fixed low-stock thresholds, from trained machine-learning forecasting, and label simulations and proposed extensions accurately.

Design Challenge:

Repeated payment callbacks may create duplicate payments or orders, while two customers may buy the last unit. Courier synchronization may fail after the order is stored. A refund must not bypass its required approval, and a support agent must not gain finance privileges merely by selecting another role in the interface.

The organization requires consistent order state, auditable refunds and dependable integration while its

small technical team initially operates a modest deployment. Additional services must be justified against operational cost and complexity.

Design and validate a requirements specification and scalable system architecture using collaborative and modeling tools of your choice that address these conditions.

Develop alternative architectural solutions and create a justified design connecting stakeholder needs, allocation rules, quality requirements and architectural decisions. Maintain traceability between requirements, UML models, architectural components and validation scenarios.

Examine at least two suitable tool arrangements for requirements tracking, collaborative documentation and UML modeling. Select a tool or combination of tools and justify its suitability, considering traceability, collaboration, version history, diagram support, accessibility and cost. Explain any limitations and how you will address them.

Validate the design against deliberately constructed scenarios. Use the findings to revise your requirements and architecture, and defend the final design's consistency, scalability and limitations. Explain how the selected tools support collaboration and keep the specification, models and design decisions consistent.

Q1 – Investigate Stakeholder Needs and Resolve Conflicts 5 Marks

Investigate the needs of customers, pickers, support agents, administrators, store managers and finance reviewers.

- Formulate at least three questions that investigate an unresolved requirement or stakeholder conflict.
- Gather findings through a small stakeholder discussion, questionnaire or process review. Where access is unavailable, use clearly labelled simulated responses.
- Identify at least three conflicting expectations.
- Develop and justify a resolution for each conflict.
- Define the system boundary, principal actors and scope exclusions.

Do not assume that every stakeholder request should become a requirement.

Evidence Expected

Investigation questions, findings or labelled simulated responses, a stakeholder analysis, conflict-resolution decisions and a justified system boundary.

Q2 – Create a Prioritized and Testable Requirements Specification 8 Marks

Create a requirements specification in a selected collaborative documentation tool and represent prioritized requirements in a selected issue tracker. Jira and Confluence are reference options; justify the arrangement chosen.

- At least twelve functional requirements and six non-functional requirements.
- Unique identifiers, sources, priorities and acceptance criteria.

- Order rules covering price and tax calculation, stock reservation, ownership, permitted status changes, dispatch deadlines, duplicate payment callbacks and refund approvals.
- Measurable quality requirements for performance, availability, security and usability.
- Explicit assumptions and unresolved questions.
- A justified prioritization method.
- Replacement of vague statements such as “checkout should be fast” with requirements that specify the operating condition and an observable acceptance threshold.
- An explanation of how at least two stakeholder conflicts influenced the final specification.

Evidence Expected

Collaborative specification, requirement records, measurable acceptance criteria, prioritization rationale and documented assumptions.

Q3 – Construct UML Models and Maintain Traceability 7 Marks

Create a consistent set of UML models covering:

- A use case diagram showing actors and principal interactions.
- A class diagram representing important domain concepts and relationships.
- A sequence diagram tracing a payment callback through order confirmation, shipment creation and courier synchronization, including a retry or failure.
- A state machine for Placed, Paid, Packed, Dispatched, Delivered and Closed orders, including authorized cancellation or return if supported.
- Approval, rejection, execution and rollback behaviour for refunds, modelled where relevant.
- A traceability matrix connecting Requirement → UML Element → Architectural Component → Validation Scenario.
- Identification and resolution of at least two inconsistencies discovered while connecting these representations.

Evidence Expected

Four UML diagrams, a traceability matrix, identified inconsistencies and documented corrections.

Q4 – Design and Justify Alternative Architectures 8 Marks

Develop two meaningfully different architectural alternatives for the system. Select appropriate patterns, such as a modular monolith, microservices or event-driven architecture. For each alternative:

- Define the principal components and responsibilities.
- Explain communication and data ownership.
- Describe how duplicate payment processing and overselling of the last unit would be prevented.
- Examine scalability, consistency, availability, security and operational complexity.

- Identify likely failure points and maintenance implications.

Then select an architecture and create component and deployment diagrams. Document at least three architectural decisions in your selected documentation tool. State each problem, alternatives, selection, justification and consequences.

Evidence Expected

two architectural alternatives, a reasoned comparison, selected component and deployment diagrams,

Q5- Validate Critical Behaviour and Revise the Design 7 marks

Define order and refund rules that must remain true throughout operation. Include invariants such as:

The same gateway event identity must not create multiple payments or order confirmations; an unapproved refund must not be marked as executed through an approval-gated operation.

Specify preconditions and postconditions for at least two critical operations. Construct at least six validation scenarios, including:

- Two customers attempting to buy the last unit of a product.
- A repeated payment callback arriving while the original callback is being processed.
- An approved refund failing at the gateway and requiring a controlled retry or rollback.
- A dispatch attempt without confirmed payment or pick confirmation.
- A courier synchronization request failing and then being retried.
- A customer or support agent attempting to approve a refund or access another customer's order.

Before validation, predict the expected behaviour and identify which requirement or design decision should enforce it.

Validate using a small prototype, executable model, structured walkthrough or another justified method. Clearly distinguish demonstrated results from predictions.

Identify a weakness and revise the relevant requirements, UML models or architecture.

Evidence Expected

Invariants, operation contracts, six validation scenarios, expected and observed behaviour, validation evidence and a documented design revision.

Q6 – Redesign for Expansion and Partial Failure 10 Marks

Changed Requirement

The organization expands from three to twelve regions. The planning workload increases from 50 to 500 simultaneous users and from 100 to 1,000 stock, payment and scan events per minute. These are design assumptions rather than demonstrated system capacity.

- Each region can define local tax, shipping and refund-approval rules.
- A disconnected warehouse scanner node must buffer uniquely identified scan and stock events for

later synchronization.

- A locally queued refund approval must not be displayed as globally approved before server validation.
- Management requires consolidated sales, stock and refund reports.
- The operating budget permits only a limited increase in infrastructure expenditure.

Create a revised requirements and architecture design that addresses the expansion.

Component	Required Work	Marks
A. Diagnose the impact	Identify affected requirements, assumptions, components and quality attributes.	2
B. Create the revised architecture	Redesign component responsibilities, data ownership, communication and deployment. Explain pending buffered events, deduplication and server-validated approval.	4
C. Validate the redesign	Construct at least three additional scenarios covering increased load, synchronization and partial failure. Provide validation evidence appropriate to your method.	2
D. Defend the design	Explain benefits, costs, consistency and availability trade-offs, and remaining limitations.	2

Do not assume that increasing the number of services automatically improves scalability. Connect each architectural change to an identified requirement or limitation.

Evidence Expected

Impact analysis, revised requirements and diagrams, synchronization rules, validation findings and an evidence-supported architectural defence.

Q7 – Individual Technical Reflection 5 Marks

Explain:

- Which requirement had the greatest influence on your architecture.
- Which stakeholder conflict was most difficult to resolve.
- Which inconsistency was discovered through modeling or validation.
- Which architectural trade-off remains unresolved or significant.
- What additional evidence would be needed before implementing the system.

Evidence Expected

A concise reflection demonstrating understanding of the created design and its limitations. Refer to your own specification, diagrams, decisions and validation findings.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Stakeholder investigation and conflict resolution	5
Q2	Prioritized and testable requirements specification	8
Q3	UML modeling and requirements traceability	7
Q4	Alternative architectures and justified design	8
Q5	Critical-behaviour validation and design revision	7
Q6	Day-4 Design Challenge – architecture redesign for expansion and partial failure	10
Q7	Individual technical reflection	5
TOTAL		50

Submission Requirements

- Assignment report covering stakeholder analysis, scope, architectural alternatives and tool selection.
- Collaborative requirements specification and issue-tracker records with unique identifiers and acceptance criteria.
- UML diagrams (use case, class, sequence, state machine, component and deployment) and the traceability matrix.
- Architectural decision records, invariants, operation contracts and validation scenarios with expected and observed behaviour.
- Day-4 revisions: impact analysis, revised requirements and diagrams, synchronization rules and validation findings.
- Individual reflection and acknowledgement of sources and tools used.

OBE Assessment Evidence

Successful completion should demonstrate the ability to analyze conflicting requirements, create a testable specification, model the system with UML, evaluate and select an architecture, validate critical behaviour, and revise the design when scale and failure conditions change.

Assessment Verification

Students may be asked to trace a requirement to its UML element, component and validation scenario, explain an order or refund invariant, justify an architectural decision, or revise the design when a requirement changes.

Overall Assessment Expectation

Investigate → Specify → Model → Architect → Validate → Redesign → Defend

The final submission must demonstrate an original, evidence-supported specification and architecture aligned with CO2, BTL 5 – Evaluate and BTL 6 – Create.

ASSIGNMENT 2 – ANSWERS

CSA10 – SOFTWARE ENGINEERING

Assignment Title: Design and Validate Requirements and Architecture for the Omnichannel Order Fulfilment Platform

Planning note: Stakeholder findings, workload figures, test outcomes and validation results are clearly labelled as simulated or proposed. They are not claims of actual interviews, production testing or measured capacity.

Q1 – INVESTIGATE STAKEHOLDER NEEDS AND RESOLVE CONFLICTS (5 MARKS)

1.1 Investigation questions

• What information must customers see to track an order without exposing payment or another customer's details?

• How should the system behave when two buyers attempt to reserve the final unit at the same time?

• Who may approve refunds, what approval threshold applies, and what evidence must be recorded?

• What should happen when a payment callback is repeated or the courier API is unavailable?

1.2 Simulated stakeholder findings

Stakeholder	Simulated need	Design implication
Customer	Fast checkout, accurate price and clear tracking.	Provide simple checkout and customer-scoped order tracking.
Picker	Accurate reserved stock and reliable pick list, including offline scanner support later.	Use atomic reservation and queue scanner events for synchronization.
Support agent	View order/return status but not card or finance data.	Apply role-based access and field-level response filtering.
Finance reviewer	Refunds above the configured limit require approval and audit evidence.	Use approval-gated refund workflow with approver, reason and timestamp.
Store manager	Regional stock, tax and shipping visibility.	Use region-aware configuration and consolidated reporting.

Administrator	Manage users, roles and system configuration.	Restrict privileged operations and record administrative audit events.

1.3 Conflicting expectations and resolutions

Conflict	**Resolution**	**Justification**
Customer wants fast checkout; inventory needs strict last-unit protection.	Use atomic stock reservation during checkout; show a short reservation wait if contention occurs.	Correct stock is more important than accepting an order that cannot be fulfilled.
Support wants complete order visibility; privacy requires restricted payment access.	Show order, shipment and return status but mask payment credentials and sensitive finance fields.	Provides support usefulness with least-privilege access.
Customer expects quick refunds; finance requires approval above a threshold.	Allow immediate request creation, but execute only after authorized approval where required.	Separates fast request handling from controlled financial execution.
Management wants consolidated reports; regions own local rules and data.	Store region identifiers and publish controlled reporting events to a central reporting view.	Enables consolidated reporting without overriding local transaction ownership.

1.4 System boundary, actors and exclusions

System boundary: The platform begins when a customer or staff member submits an order and ends after delivery or an authorized return/refund is recorded. It includes catalogue/checkout, order management, inventory reservation, payment adapter, warehouse picking, shipment/courier adapter, tracking, refunds, support lookup, user authorization and reporting.

Principal actors: Customer, picker, support agent, finance reviewer, store manager, administrator, payment gateway and courier API.

Exclusions: Real card-data storage, direct control of courier vehicles, production tax/legal advice, machine-learning demand forecasting, and full ERP replacement. External services are represented by controlled test interfaces if live access is unavailable.

1.5 Scope decision

The representative release uses synthetic products, orders and stock events. It prioritizes reliable order, payment, reservation, dispatch and refund controls. Advanced analytics and forecasting are deferred because they do not resolve the immediate consistency and authorization risks.

Q2 – CREATE A PRIORITIZED AND TESTABLE REQUIREMENTS SPECIFICATION (8 MARKS)

2.1 Collaborative tools and prioritization

Tool arrangement	Strengths	Limitations / mitigation
Jira + Confluence	Jira tracks requirement IDs, priorities, owners, dependencies and test issues; Confluence stores the specification, decision records and change history.	May require paid access and configuration; use a small project template and link each Jira issue to its Confluence requirement.
GitHub Issues/Projects + Markdown repository + PlantUML	Version-controlled requirements, issue tracking and text-based UML diagrams; accessible to developers and supports review history.	Less structured requirements reporting; use labels, templates and a traceability table.

Selected arrangement: Jira + Confluence, with PlantUML or diagrams.net for UML. Jira provides prioritized work records; Confluence provides the controlled specification and architecture decision records. Requirement IDs are repeated in Jira issue titles and linked to model elements and tests.

Prioritization: MoSCoW. Must-have requirements protect order integrity, payment idempotency, privacy and refund authorization. Should-have requirements support tracking, regional rules and reporting. Could-have items include advanced analytics. Won't-have items include real card-data storage and ML forecasting in this release.

2.2 Functional requirements

ID / Source	Requirement and priority	Acceptance criteria
FR01 Customer	The system shall create an order with a unique order ID. Must.	Given valid cart items, when checkout is submitted, then one order record with item and price snapshot is persisted.

FR02 Customer / Finance	The system shall calculate item totals, configured tax and shipping. Must.	For a defined test cart and region, the stored total equals item subtotal + configured tax + shipping - valid discounts.
FR03 Picker / Manager	The system shall reserve available stock atomically. Must.	When two requests compete for the final unit, only one reservation succeeds and available stock never becomes negative.
FR04 Customer	The system shall store payment attempts against an order. Must.	Each attempt has a unique provider reference, status and timestamp linked to one order.
FR05 Gateway	The system shall process duplicate payment callbacks idempotently. Must.	Repeated callbacks with the same event ID create no second payment or order confirmation.
FR06 Picker	The system shall generate a pick list from confirmed reservations. Must.	Only reserved order lines appear, with SKU and required quantity.
FR07 Picker	The system shall permit dispatch only after payment confirmation and pick confirmation. Must.	Dispatch requests missing either condition are rejected and audited.
FR08 Customer / Support	The system shall display order and shipment status. Must.	The authorized customer sees only their order and the latest persisted status.
FR09 Courier	The system shall synchronize shipment updates with retry. Should.	Failed courier calls are queued and retried; repeated updates do not duplicate shipment records.
FR10 Customer	The system shall accept return requests for eligible delivered orders. Should.	A return request stores order, items, reason and status; ineligible states are rejected.
FR11 Finance	The system shall enforce refund approval rules. Must.	Refund execution is blocked until required authorized approval is stored.

FR12 Finance / Admin	The system shall maintain an audit trail for payments, refunds and stock corrections. Must.	Each record stores actor, action, timestamp, reason and linked transaction/order.
FR13 Support	The system shall provide role-filtered order lookup. Must.	Support users can see order/return/shipment fields but cannot retrieve payment credentials or approve refunds.
FR14 Manager	The system shall provide consolidated sales, stock and refund summaries. Should.	Reports aggregate synthetic transactions by region and date and identify their data refresh time.

2.3 Non-functional requirements

ID	Quality requirement	Acceptance threshold / verification
NFR01 Performance	Checkout response time.	Under 2 seconds at the 50-user baseline for 95% of synthetic requests, excluding external gateway delay.
NFR02 Performance / scale	Expansion workload target.	Design target of 500 simultaneous users and 1,000 stock/payment/scan events per minute; validate using a load test, not assumed capacity.
NFR03 Availability	Core order service availability.	Target 99.5% monthly availability for the representative release, excluding planned maintenance.
NFR04 Security	Authentication and authorization.	All protected endpoints require authenticated identity and server-side role checks; negative tests reject unauthorized refund and cross-customer access.
NFR05 Privacy	Sensitive data minimization.	No real card data is stored; support API responses exclude payment credentials and restricted finance fields.

NFR06 Usability	Core task completion.	At least 90% of five representative test users complete order lookup or picking task without assistance in a usability walkthrough.

2.4 Order rules, assumptions and unresolved questions

• Price/tax: Store the item-price snapshot and apply the tax/shipping configuration associated with the order region at checkout; later configuration changes do not silently rewrite an existing order.

• Stock: Reservation is atomic and has a defined expiry; stock correction requires an authorized role and an audit reason.

• Ownership/status: The customer owns their order view; only permitted roles can change order state. Valid transitions are enforced server-side.

• Dispatch: Dispatch requires confirmed payment and completed pick confirmation. A proposed operational target is dispatch within one business day after payment and pick completion.

• Duplicate callbacks: Persist gateway event identity with a unique constraint and process it transactionally; repeated event IDs return the previously recorded outcome.

• Refund: Refund request, approval and execution are separate states. Refunds above the configured regional threshold require a finance reviewer; an unapproved refund cannot be executed.

• Assumptions: Synthetic data only; gateway/courier APIs may be unavailable; one modest deployment initially; 50 concurrent users is the baseline planning workload.

• Unresolved: Exact tax rules by jurisdiction, refund threshold, reservation expiry, courier service-level agreement, retention period and legal audit requirements require stakeholder confirmation.

Stakeholder conflict influence: Support visibility is constrained by NFR04/NFR05, so support receives status data but not payment credentials. Checkout speed is balanced against FR03 by reserving stock atomically rather than accepting an unverified last-unit order.

Q3 – CONSTRUCT UML MODELS AND MAINTAIN TRACEABILITY (7 MARKS)

The following UML-style models represent the proposed design. They are planning diagrams based on the requirements, not screenshots from a live modeling tool.

3.1 Use case diagram

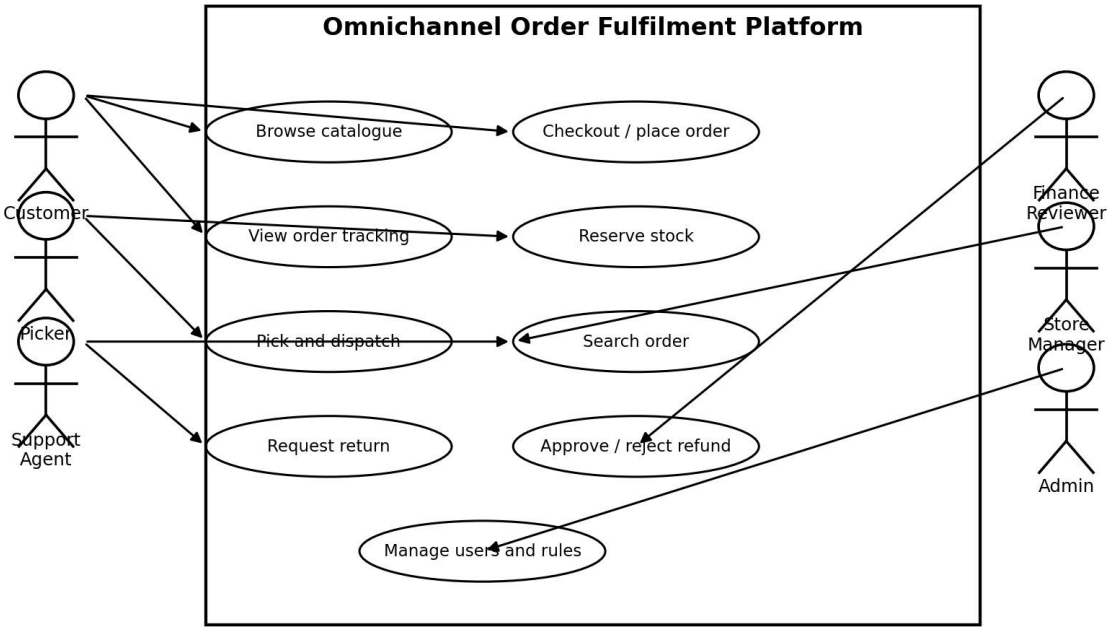


Figure 1. Principal actors and their interactions with the order fulfilment platform.

3.2 Class diagram

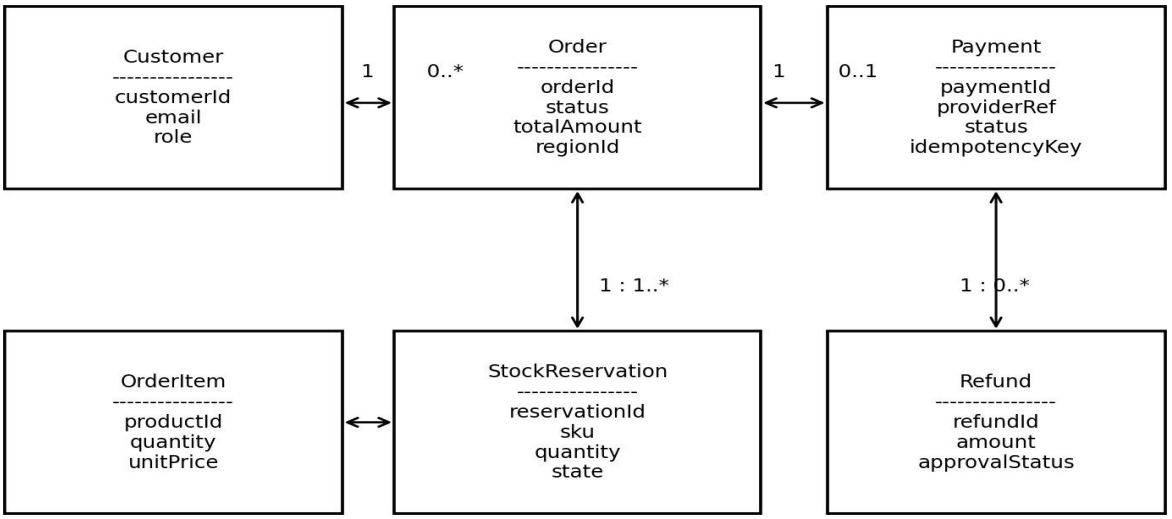


Figure 2. Core domain classes and relationships.

3.3 Sequence diagram – payment callback and courier retry

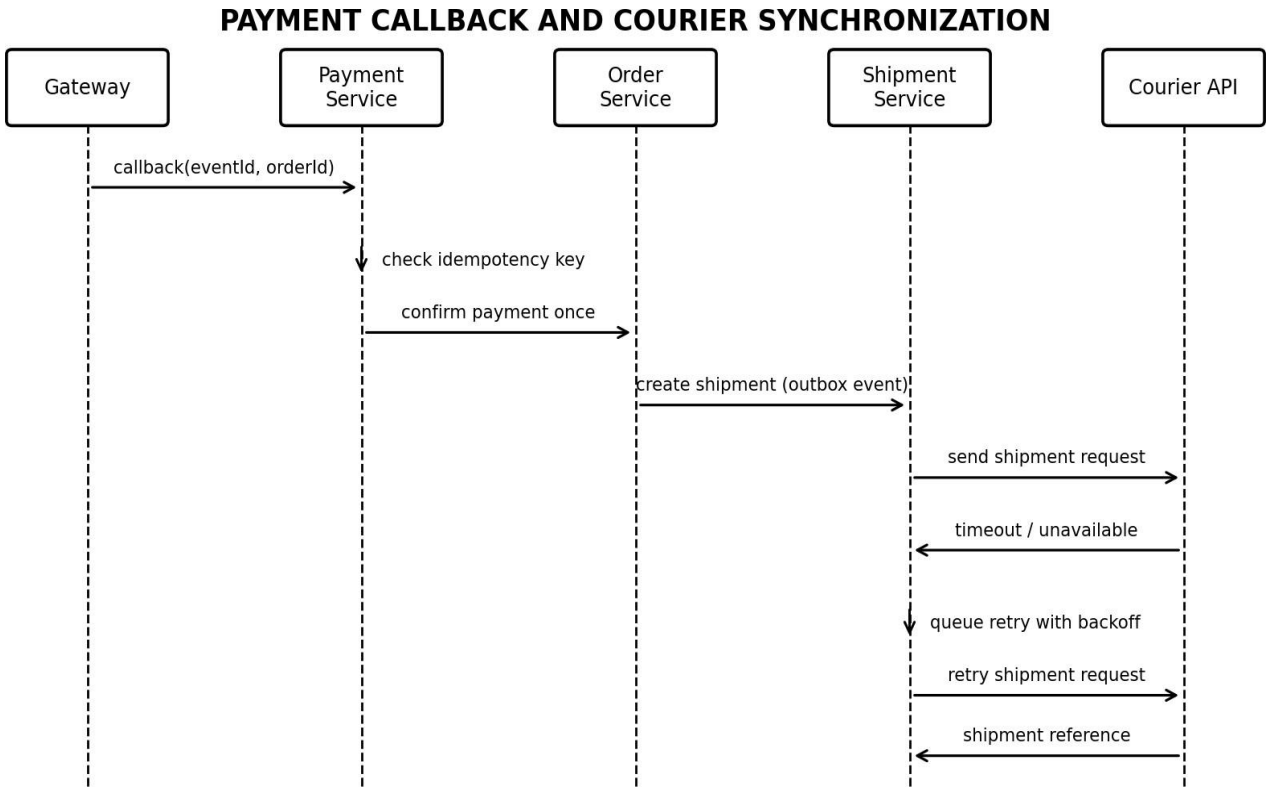


Figure 3. Idempotent payment callback followed by shipment creation and courier retry.

3.4 Order state machine

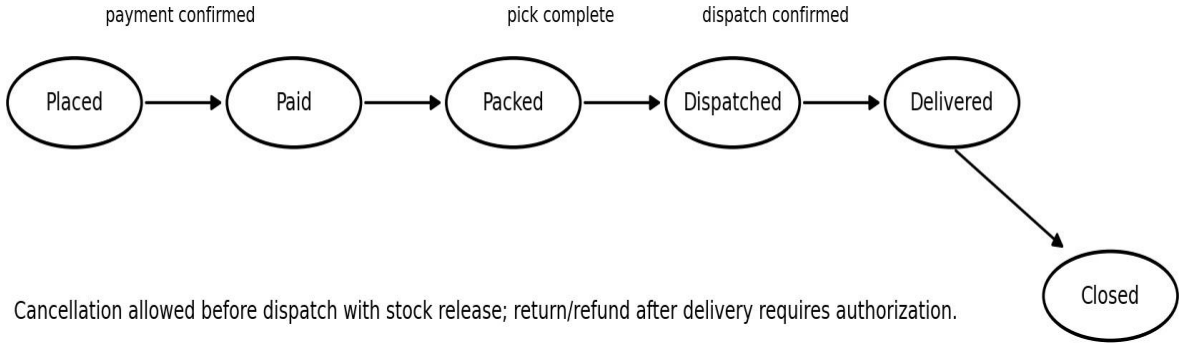


Figure 4. Order states; cancellation and return/refund are controlled side processes.

3.5 Traceability matrix

Requirement	UML element	Architecture component	Validation scenario
FR01 / FR02	Checkout use case; Order class	API Gateway, Order Module	Create order and verify stored price/tax total.
FR03	Reserve stock use case; StockReservation class	Inventory Module + PostgreSQL transaction	Two buyers compete for final unit.
FR05	Payment callback sequence; Payment class	Payment Module + Inbox/idempotency table	Duplicate callback arrives concurrently.
FR06 / FR07	Picker use case; order state machine	Inventory and Order Modules	Dispatch without payment or pick confirmation.
FR09	Sequence retry path	Shipment Adapter + worker queue	Courier API unavailable then recovers.
FR11 / FR12	Refund use case; Refund class	Refund Module + audit log	Unapproved refund execution is rejected.
FR13 / NFR05	Support lookup use case	API authorization and response filtering	Support attempts cross-customer or payment-data access.

3.6 Modelling inconsistencies found and corrections

Inconsistency found	Correction
Initial sequence implied shipment creation immediately after payment callback, even if order state was not committed.	Added transactional outbox: shipment event is published only after payment and order state are durably committed.
Initial class view did not distinguish refund request from approval and execution.	Refund class now includes approval status; workflow separates Requested, Approved/Rejected and Executed/Failed.
Initial tracking use case risked exposing payment fields to support.	Added role-filtered support response and server-side authorization requirement.

Q4 – DESIGN AND JUSTIFY ALTERNATIVE ARCHITECTURES (8 MARKS)

4.1 Alternative A – Modular monolith

A single deployable application contains separate Order, Inventory, Payment, Shipment, Refund, Support and Reporting modules. Modules communicate through internal interfaces and share one PostgreSQL database with clear schema/table ownership. A transactional outbox and inbox tables support reliable event delivery and deduplication.

4.2 Alternative B – Microservices with event-driven integration

Order, Inventory, Payment, Shipment, Refund and Reporting run as independent services with service-owned databases. They communicate through APIs and a message broker. Each service can scale independently, but distributed transactions, observability, deployment and data consistency become more complex.

Criterion	Modular monolith	Microservices / event-driven
Communication	In-process module interfaces; async outbox workers for external systems.	Network APIs and message broker between services.
Data ownership	One database with module-level ownership and controlled access.	Each service owns its database; data replicated through events.
Duplicate payment	Unique event ID and idempotency key in Payment module transaction.	Payment service inbox/deduplication store and idempotent consumer.

Overselling	Atomic conditional stock update/row lock in one database transaction.	Inventory service serializes reservations or uses atomic conditional update.
Scalability	Scale application instances together; simpler at modest load.	Independent service scaling; better isolation at high demand.
Availability	Fewer network dependencies; one application failure can affect all modules.	Service isolation possible, but broker/network failures and partial availability must be handled.
Security	Central authentication and module-level authorization.	Service-to-service identity, API gateway and distributed policy management.
Operations and cost	Lower deployment and monitoring overhead; easier for small team.	Higher infrastructure, deployment, tracing and on-call complexity.

Selected architecture: Modular monolith for the initial release, with clear module boundaries, asynchronous outbox workers and interfaces that permit later extraction. It meets the stated modest initial deployment without assuming microservices automatically improve scalability. If measured regional load later exceeds the monolith's limits, high-load modules such as reporting or shipment integration can be separated.

4.3 Component diagram

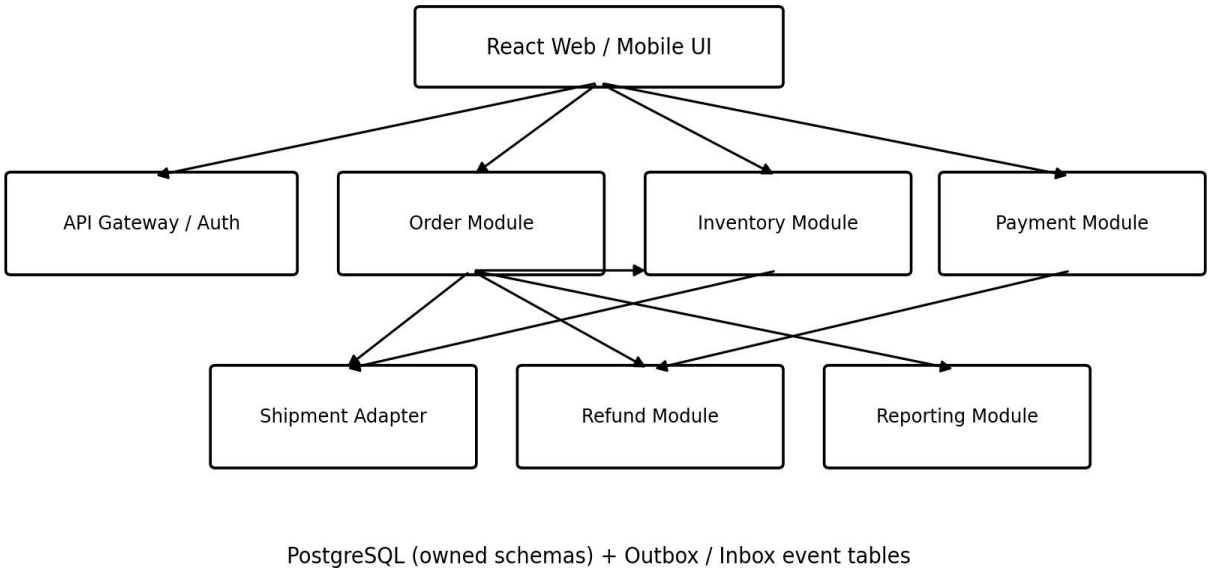


Figure 5. Selected modular-monolith component architecture.

4.4 Deployment diagram

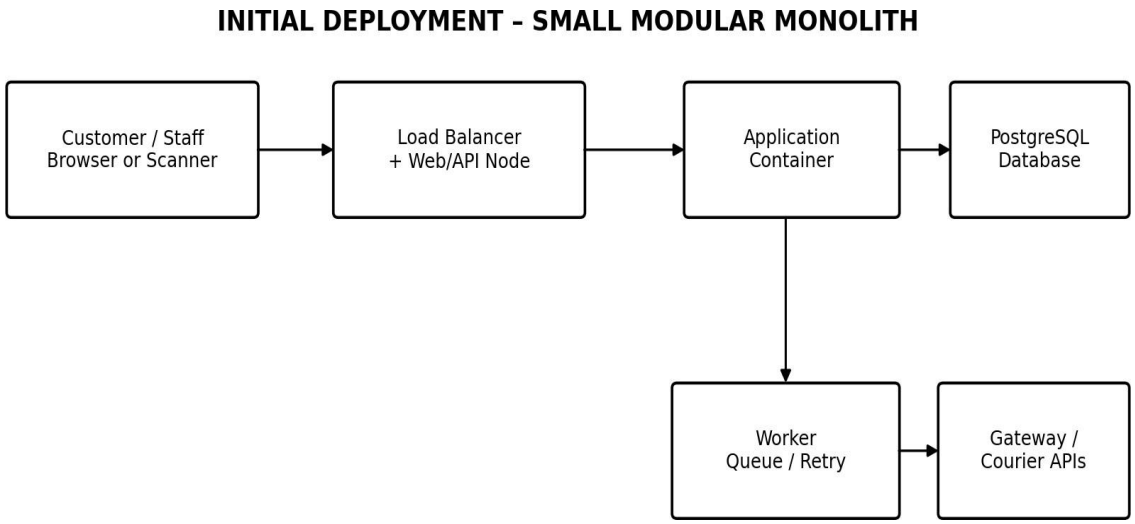


Figure 6. Initial deployment with application node, database and background integration worker.

4.5 Architectural decision records

ADR	**Problem and alternatives**	**Decision, reason and consequence**
ADR-01: Architecture style	Need modest initial operations but clear future scaling. Alternatives: monolith or microservices.	Choose modular monolith. Lower cost and operational overhead; modules must maintain strict boundaries to avoid a tightly coupled codebase.
ADR-02: Payment callback safety	Gateway may repeat callbacks or time out. Alternatives: process every callback or deduplicate persistently.	Use unique gateway event ID, idempotency key and transactional inbox. Prevents duplicate payment state; requires retention and reconciliation policy.
ADR-03: Reliable external synchronization	Courier API may fail after order commit. Alternatives: synchronous-only call or durable outbox/worker retry.	Use transactional outbox, retry with backoff and dead-letter/manual review. Adds worker and monitoring complexity but avoids lost shipment events.

Q5 – VALIDATE CRITICAL BEHAVIOUR AND REVISE THE DESIGN (7 MARKS)

5.1 Invariants

• INV01: One gateway event identity can produce at most one successful payment record and one order confirmation.

• INV02: Available stock must never become negative; at most one reservation can consume the final available unit.

• INV03: Dispatch is not permitted unless payment is confirmed and picking is complete.

• INV04: A refund cannot enter Executed unless the required authorized approval is recorded.

• INV05: Every payment, refund, stock correction and order-state change is traceable to an actor and timestamp.

• INV06: A customer or support agent cannot access another customer's restricted order or payment data.

5.2 Operation contracts

Operation	Preconditions	Postconditions
reserveStock(orderId, SKU, quantity)	Order exists; quantity > 0; caller is authorized; available stock is sufficient.	Exactly one reservation is created; available stock decreases by quantity; if insufficient, no stock change occurs.
executeRefund(refundId, actor)	Refund request exists; required approval is Approved; actor/service is authorized; payment is refundable.	Gateway refund attempt is recorded; success sets Executed with provider reference; failure remains Failed/Pending Retry and does not falsely report success.

5.3 Validation scenarios – structured walkthrough

Method: Structured walkthrough against the proposed rules and component responsibilities. The results below are simulated expected/observed outcomes from a logical design review, not execution against a running production system.

Scenario	Predicted behaviour / enforcing rule	Simulated walkthrough result
V1: Two customers buy the last unit.	Atomic conditional stock reservation permits one winner; FR03 / INV02.	Pass: one reservation succeeds; second receives unavailable response; stock remains zero, not negative.
V2: Same payment callback arrives twice concurrently.	Unique gateway event ID and idempotent transaction; FR05 / INV01.	Pass: one payment record and one confirmation; duplicate receives recorded outcome.
V3: Approved refund fails at gateway.	Keep refund Failed/Pending Retry; retry with same idempotency key; do not mark Executed; FR11 / INV04.	Pass: audit record shows failure and retry state; no false success state.
V4: Dispatch attempted before payment or picking.	Server validates order state and required preconditions; FR07 / INV03.	Pass: dispatch rejected and action logged.
V5: Courier synchronization fails, then recovers.	Outbox event remains durable; worker retries with backoff and deduplicates shipment update; FR09.	Pass: local order remains stored; shipment sync eventually updates after retry.
V6: Support/customer attempts unauthorized access or refund approval.	Server-side role and ownership checks; FR13 / NFR04 / INV06.	Pass: request rejected; restricted data is not returned; audit/security event recorded.

5.4 Weakness and design revision

Weakness: The first design described retry behaviour but did not define how repeated worker delivery would be deduplicated. Revision: add a unique event ID to every outbox message and a consumer inbox record with a unique constraint. Shipment updates become idempotent. The sequence diagram and FR09 acceptance criteria are updated to require safe replay and a dead-letter/manual-review path after the retry limit.

Q6 – REDESIGN FOR EXPANSION AND PARTIAL FAILURE (10 MARKS)

6.1 A. Diagnose the impact

Changed condition	Affected design area	Impact
Three to twelve regions	Tax, shipping, refund rules and reporting	Need region-scoped configuration, effective dates and regional approval matrix.
50 to 500 simultaneous users	API, database connections and application capacity	Need horizontal application scaling, connection pooling, caching of read-only catalogue/configuration data and load testing.
100 to 1,000 events/minute	Stock/payment/scan event processing	Need asynchronous queues, batching where safe, idempotent consumers and monitoring.
Disconnected scanner node	Warehouse event capture	Need local durable event buffer, unique event IDs, timestamps and later synchronization.
Locally queued refund approval	Refund authorization	Local state must remain Pending Server Validation; it cannot display globally Approved until server authorization succeeds.
Limited infrastructure budget	Deployment topology	Prefer scaling the existing modular monolith and workers before extracting multiple services.

6.2 B. Revised architecture and data ownership

Retain the modular monolith initially, but separate the web/API tier from background workers and add region-aware configuration, durable event ingestion and reporting read models. Deploy multiple stateless application instances behind a load balancer. Keep PostgreSQL as the authoritative transaction store with

connection pooling, indexed region/order keys and tested backup/restore. Add a durable queue only where asynchronous integration or scanner synchronization requires it.

Component	Revised responsibility / ownership
Order module	Owns order state, order lines, region ID, price/tax/shipping snapshot and permitted transitions.
Inventory module	Owns stock balances and reservations; applies atomic reservation and stock correction controls.
Payment/refund modules	Own payment attempts, gateway event identities, refund request/approval/execution and audit evidence.
Regional configuration	Owns versioned region tax, shipping and refund threshold rules; order stores the applied rule version.
Scanner synchronization service	Accepts uniquely identified scan events, stores them in a durable local buffer and syncs to server when connected.
Reporting read model	Consumes validated events and builds consolidated regional sales, stock and refund summaries; reports show refresh time.

6.3 Offline scanner and refund synchronization rules

• Each scanner event receives a globally unique event ID, device ID, region ID, event type, local timestamp and sequence number.

• While offline, the scanner writes events to durable local storage and marks them Pending Sync; it must not claim server-side stock adjustment is complete.

• On reconnection, events are uploaded in batches. The server validates device identity, region, event ID, sequence and current stock rules before applying them.

• Server inbox uniqueness prevents duplicate replay. Accepted, rejected and conflict outcomes are returned to the scanner; conflicts require authorized review rather than silent overwrite.

• Offline refund approval is only a locally queued request. The UI displays Pending Server Validation until the server verifies reviewer identity, region threshold and refund state.

• Consolidated reports consume server-validated events and display data freshness; pending local events are not counted as approved refunds or confirmed stock movements.

6.4 Revised component diagram

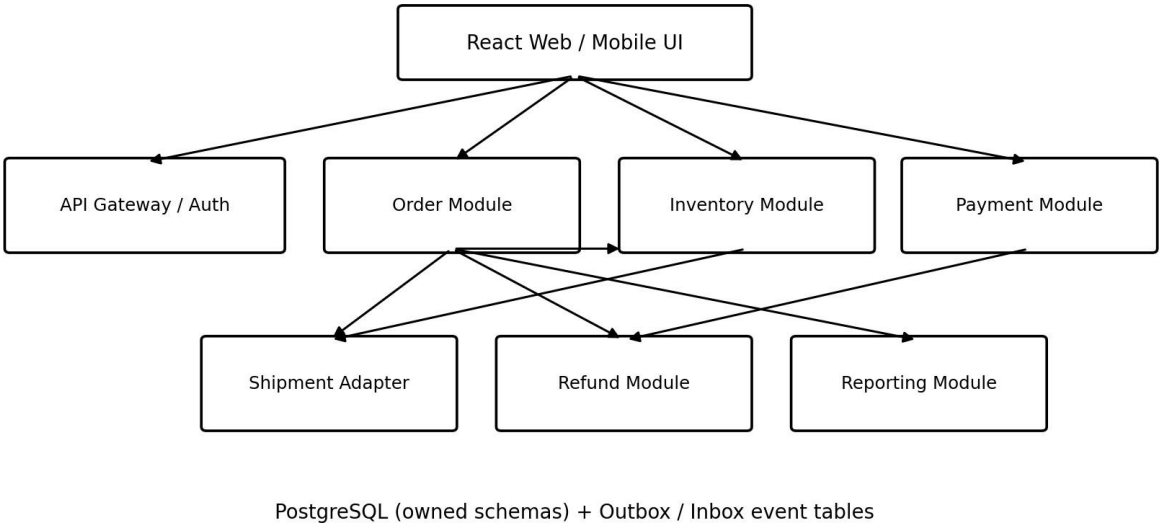


Figure 7. Core modular components retained; regional configuration, scanner sync and reporting read model are added as bounded modules/services.

6.5 C. Additional validation scenarios

Scenario	**Expected control**	**Simulated design-review result**
V7: Load rises to 500 concurrent users.	Horizontal stateless API instances, connection pooling, cache and load test at target.	Design review: capacity is a target, not proven. Load test and database saturation monitoring required before launch.
V8: Scanner is offline for 30 minutes and reconnects with duplicate events.	Durable local buffer, unique IDs, server inbox deduplication and conflict response.	Pass in logical walkthrough: duplicate event is applied once; unresolved stock conflict is flagged.
V9: Local refund approval queued while network is disconnected.	UI remains Pending Server Validation; server validates identity and regional threshold before Approved.	Pass: no global approval or gateway refund execution occurs while offline.
V10: Queue/worker stops after order commit.	Outbox remains durable; worker resumes and retries after restart.	Pass: event is not lost; monitoring alerts on queue age and retry exhaustion.

6.6 D. Defence, trade-offs and limitations

The revised design supports regional configuration and higher demand while controlling cost by scaling stateless application instances and background workers before adopting a full microservices platform. Strong consistency is retained for order, stock reservation, payment and refund approval in PostgreSQL transactions. Eventual consistency is accepted for courier updates, scanner synchronization and reporting. During a network partition, local scanner events can be captured but are not treated as globally committed until server validation.

The design increases availability of user-facing reads through multiple application instances, but PostgreSQL remains a critical dependency and requires backup, failover and monitoring. A durable queue adds operational cost and must be monitored. If load testing identifies database bottlenecks, inventory or reporting may be extracted selectively. Remaining limitations include unknown regional legal rules, offline device security, conflict resolution complexity and the need for measured performance testing.

Q7 – INDIVIDUAL TECHNICAL REFLECTION (5 MARKS)

The requirement with the greatest influence was preventing duplicate payment callbacks and overselling the final unit. These failures can create financial loss, incorrect stock and customer disputes, so the architecture uses persistent idempotency records and atomic stock reservation.

The most difficult stakeholder conflict was balancing support visibility with customer privacy. The resolution was to provide support with order, return and shipment status while blocking payment credentials and finance-only actions through server-side authorization.

Modeling exposed an inconsistency: shipment creation appeared to occur directly after a callback without guaranteeing that the order transaction had committed. The design was corrected by using a transactional outbox and retryable shipment worker. A second correction separated refund request, approval and execution states.

The most significant remaining trade-off is strong consistency for critical financial/stock operations versus eventual consistency for courier tracking, scanner synchronization and reporting. Before implementation, the team needs verified regional tax/refund rules, gateway and courier contracts, security review, offline-device threat analysis, accessibility testing and measured load-test results.

CONCLUSION

The proposed design translates stakeholder needs into prioritized, measurable requirements and traceable UML models. A modular monolith with explicit module ownership, atomic stock reservation, idempotent payment callbacks, approval-gated refunds and transactional outbox processing is suitable for the modest initial deployment. Structured validation identifies and corrects weaknesses in shipment retries and refund state handling. The expansion design adds regional configuration, worker scaling and offline scanner synchronization while preserving server authority for stock and refund decisions.

REFERENCES

1. Mucha, J., Kaufmann, A., & Riehle, D. (2024). A systematic literature review of pre-requirements specification traceability. Requirements Engineering, 29, 119–141.

2. Koboyatshwene, T., & Ayalew, Y. (2025). Requirements traceability: A systematic literature review. Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, 1509–1513.

3. Cederbladh, J., Cicchetti, A., & Suryadevara, J. (2024). Early validation and verification of system behaviour in model-based systems engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology, 33(3), Article 81.

4. Umar, M. A., & Lano, K. (2024). Advances in automated support for requirements engineering: A systematic literature review. Requirements Engineering, 29, 177–207.

5. Razzaq, A., et al. (2023). A systematic mapping study: The new age of software architecture from monolithic to microservice architecture—awareness and challenges. Computer Applications in Engineering Education, 31(2), 421–451.

6. Wang, L., Jiang, Y. X., Wang, Z., et al. (2023). The operation and maintenance governance of microservices architecture systems: A systematic literature review. Journal of Software: Evolution and Process, 35(10), e2433.

7. Söylemez, M., Tekinerdogan, B., & Tarhan, A. K. (2022). Challenges and solution directions of microservice architectures: A systematic literature review. Applied Sciences, 12(11), 5507.

Screenshot :

